Azure RMS / AIP → Built-in Unified Labeling Migration — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Azure RMS / AIP → Built-in Unified Labeling Migration (Microsoft Purview) 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 5

Scope boundary . . . 6

Outcome . . . 6

1. Solution Overview — Azure RMS / AIP → Built-in Unified Labeling Migration (Microsoft Purview) 6

What this solution is . . . 6

Why this matters . . . 7

How it works . . . 7

Parity dimensions (what "parity" covers) . . . 8

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 9

The standard deployment model . . . 9

2. Requirements Guide — Azure RMS / AIP → Built-in Unified Labeling Migration 9

2.1 Sensitivity-labeling licensing (labels + encryption) . . . 10

2.2 Least-privilege information-protection roles . . . 10

2.3 Toolchain: S&C; PowerShell + AIPService module + Graph . . . 10

2.4 Azure Rights Management (protection) is activated . . . 10

2.5 Supported Office builds for built-in labeling . . . 11

2.6 Current labeling estate inventoried (labels, policies, templates) . . . 11

2.7 Labels present in the unified store (no classic-only labels) . . . 11

2.8 Azure RMS super-users configured (recovery) . . . 11

2.9 Office client-policy delivery mechanism . . . 11

2.10 Stakeholders, pilot ring, and change control . . . 12

Checklist . . . 12

Global Requirements — Microsoft Purview Solutions 12

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 13

2. Purview roles (least-privilege) and portal access . . . 13

3. Security & Compliance PowerShell + Graph toolchain . . . 13

4. Data sources & scope in scope . . . 14

5. Compliance, records, privacy & legal stakeholders . . . 14

6. A pilot scope and change control . . . 14
3. Implementation Guide — Azure RMS / AIP → Built-in Unified Labeling Migration 15
Step 1 — Inventory the current estate (Assess) . . . 15
Step 2 — Build the parity map (Design) . . . 15
Step 3 — Configure label-policy settings for parity (Configure, preview-first) . . . 16
Step 4 — Confirm labels and protection in the unified store (Configure) . . . 16
Step 5 — Stage and cut over the pilot ring (Pilot) . . . 17
Step 6 — Validate parity on the ring (Pilot → Validate) . . . 17
Step 7 — Expand tenant-wide (Expand) . . . 17
5. Testing Guide — Azure RMS / AIP → Built-in Unified Labeling Migration 18
Test 1 — Inventory is complete and matches the portal . . . 18
Test 2 — Azure RMS is activated and recoverable . . . 18
Test 3 — Label-policy settings dry-run makes no changes . . . 18
Test 4 — Parity settings resolve after commit . . . 18
Test 5 — Every label resolves for a pilot user . . . 18
Test 6 — Built-in labeling is active and the add-in is gone (ring) . . . 19
Test 7 — Encryption and custom permissions behave as before (ring) . . . 19
Test 8 — Content protected by the old add-in / templates still opens . . . 19
Test 9 — Parity report shows no open protection gap . . . 19
Pass/fail gate . . . 19
6. Troubleshooting Guide — Azure RMS / AIP → Built-in Unified Labeling Migration 19
After cutover, a label the add-in showed is missing from the Office ribbon . . . 20
`Connect-AipService` fails or `Get-AipServiceConfiguration` errors . . . 20
Azure RMS `FunctionalState` is `Disabled` . . . 20
Encryption/custom permissions behave differently than under the add-in . . . 20
A departmental RMS template's protection didn't carry over . . . 20
The AIP add-in is still loading after you applied the client policy . . . 21
`Set-LabelPolicy -AdvancedSettings` rejects a key or has no effect . . . 21
Old protected content won't open on a migrated device . . . 21
`Execute-AzureADLabelSync` runs but protection still lags . . . 21
7. Best Practices — Azure RMS / AIP → Built-in Unified Labeling Migration 22
8. Deliverables Summary — Azure RMS / AIP → Built-in Unified Labeling Migration 23
Documentation . . . 23
PowerShell scripts . . . 23
What the buyer receives . . . 24
Outcome . . . 24

Executive Summary — Azure RMS / AIP → Built-in Unified Labeling Migration (Microsoft Purview)

The **Azure Information Protection (AIP) add-in for Office** is being retired. Microsoft has disabled it by default and moved to **built-in labeling** — the sensitivity-labeling experience built directly into Word, Excel, PowerPoint, and Outlook, no add-in required. Organizations still on the add-in face a compliance gap (labeling on a client at end of life) and a data-risk exposure (protection that may stop working). This solution moves you off the AIP add-in, and off legacy **Azure Rights Management (RMS) departmental templates**, to **built-in unified labeling** in Microsoft Purview — **with policy parity**, so the labels, encryption, and behaviour your users see after the cutover match what they had before.

It gives you a repeatable, end-to-end migration: inventory the current labels, label policies, and RMS templates; build a **parity map** from every AIP add-in feature and departmental template to its built-in equivalent; tune **label-policy advanced settings** for behavioural parity; cut a **pilot ring** over in a change window; validate parity against explicit pass/fail gates; then expand tenant-wide and decommission the add-in — using only the Microsoft Purview portal, Security & Compliance PowerShell, the AIPService protection module, Microsoft Graph, and Office client policy.

This is a complete, deploy-it-today package for a competent M365 / information-protection administrator running the migration.

What you get

• A plain-language Solution Overview of the AIP add-in retirement, the move to built-in labeling with parity, legacy RMS/template mapping, and the parity dimensions.

• A Requirements Guide with verify / implement-if-missing for every prerequisite — licensing, roles, the S&C PowerShell + AIPService + Graph toolchain, RMS activation, Office builds, and the client-policy delivery mechanism.

• An Implementation Guide of precise, ordered portal + PowerShell + client-policy steps, applied pilot-first, with each label, feature, template, and policy named.

• Five PowerShell scripts: estate inventory, a label-policy advanced-settings helper (`-WhatIf` by default), a parity check, a migration/parity report, and an RMS/scope-health helper.

• Testing and Troubleshooting guides for a confident, safe cutover.

What this delivers

An inventoried estate; a parity map that leaves no label, feature, or template unaccounted for; label-policy settings that reproduce the add-in behaviour; a pilot ring cut over to built-in labeling with the add-in gone; encryption, custom permissions, and departmental-template protection reproduced; previously protected content still opening; and a validated, evidenced expansion tenant-wide.

The deployment model

Assess → Design → Configure → Pilot → Validate → Expand — with high-impact cutover changes piloted on a ring and validated before enforcement.

Scope boundary

This is a **migration**, not a taxonomy or auto-labeling project. Designing new labels, authoring auto-labeling or DLP, and standing up the Information Protection scanner are separate Purview solutions. Feature availability and minimum Office builds are licensing-/build-dependent and cited or flagged "verify your version/licensing," never invented. Whether a legacy departmental template still applies is a records/compliance determination, deferred to the organization's compliance, records, privacy, and legal functions — this solution provides the technical method and evidence, not the determination.

Outcome

Off the retiring AIP add-in and legacy RMS templates, onto built-in unified labeling in Microsoft Purview — with parity, piloted, validated, and defensible.

1. Solution Overview — Azure RMS / AIP → Built-in Unified Labeling Migration (Microsoft Purview)

What this solution is

The **Azure Information Protection (AIP) add-in for Office** is on its way out — Microsoft has disabled it by default and is retiring it in favour of **built-in labeling**, the sensitivity-labeling experience baked directly into Word, Excel, PowerPoint, and Outlook with no add-in required. Leaving users on the AIP add-in past its end of life is both a **compliance gap** (labeling on a dead client) and a **data-risk exposure** (protection that may stop working). This solution moves you off the AIP add-in and off legacy **Azure Rights Management (RMS) departmental templates** to **built-in unified labeling** in Microsoft Purview — **with policy parity**, so the labels, protection, and behaviour your users see after the cutover match what they had before.

- Inventory the current state: **sensitivity labels** and **label policies** in Microsoft Purview, the **Azure RMS / protection templates** in the protection service, and where the **AIP add-in** is still in use.
- Confirm the **built-in labeling prerequisites** (supported Office builds, encryption / co-authoring requirements) and that the **Azure Rights Management** protection service is activated.
- Build a **parity map**: every AIP add-in feature and every departmental RMS template mapped to its built-in-labeling / sensitivity-label equivalent, with gaps flagged.
- Configure Office to **use built-in labeling and disable the AIP add-in**, and tune **label-policy advanced settings** so behaviour matches the old client.
- **Pilot on a ring first**, validate parity against pass/fail gates, then **expand** tenant-wide.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`), the **AIPService** PowerShell module (the protection service),

Microsoft Graph, and Office client policy (Group Policy / Microsoft 365 Apps admin center) — no third-party tooling.

> ***This is a migration, not a label-taxonomy build.*** *It moves an *existing* labeling estate from the AIP add-in / legacy RMS onto built-in labeling with parity. Designing a label taxonomy from scratch, authoring auto-labeling policies, or standing up the Information Protection scanner are separate Purview solutions. Where this guide reaches the edge of one of those, it points to it rather than duplicating it.*

Why this matters

- **The AIP add-in is retiring — verify the current status for your tenant.** Microsoft has disabled the

AIP unified labeling add-in for Office by default and is retiring it; built-in labeling is the supported path. Users left on the add-in risk a labeling/protection experience that degrades or stops. Confirm the exact retirement state against Microsoft's current guidance and your Office channel.

- **Parity is the whole point.** A migration that silently drops a feature (custom permissions, a

default Outlook label, a mandatory-labeling prompt, a protected-PDF behaviour) is a migration that creates help-desk tickets and, worse, unprotected data. This solution treats **feature-and-template parity** as the deliverable, not an afterthought.

- **Legacy RMS templates don't move themselves.** Azure RMS departmental/protection templates and

sensitivity-label encryption are two views of the same protection service, but templates are **not** auto-converted into labels. Each must be mapped to a label with matching encryption settings — the *technical* method is here; which template still applies is a records/compliance determination.

- **High-impact change deserves a pilot.** Flipping the labeling client for a user changes how every

document and email is classified and protected. Doing it to a **pilot ring in a controlled window** first — and validating parity — is the difference between a clean cutover and a tenant-wide incident.

How it works

1. **Assess.** Read-only: inventory sensitivity labels, label policies (and their advanced settings),

Azure RMS activation state, protection/RMS templates, and signals of where the AIP add-in is still in use. No changes.

1. **Design.** Build the **parity map** — AIP add-in features → built-in equivalents, and departmental

RMS templates → sensitivity labels with matching encryption — and record the target **Office client policy** (use built-in labeling, disable the AIP add-in) and the **label-policy advanced settings** needed for behavioural parity.

1. **Configure.** Apply the label-policy advanced settings (`-WhatIf` preview first), confirm the labels

and protection exist in the unified store, and stage the Office client policy — all scoped to the pilot ring.

1. **Pilot (cutover the ring).** Move the **pilot ring** to built-in labeling / disable the add-in in a

change window; keep everyone else on the current experience.

1. **Validate.** Confirm on the ring that every label appears, applies, and protects exactly as before, that departmental-template protection is reproduced, and that no parity gap is open — against explicit pass/fail gates.

1. **Expand (enforce).** Broaden the Office client policy from the ring to the full population, monitor, and decommission the AIP add-in.

Parity dimensions (what "parity" covers)

Dimension	AIP add-in / legacy RMS	Built-in labeling target
Label list & order	Labels shown by the add-in	Same labels from the Purview label policy
Default / mandatory labeling	Add-in policy settings	Label-policy settings + advanced settings
Encryption & permissions	Label encryption / custom permissions	Sensitivity-label encryption (parity; some features need newer Office)
Departmental protection	Azure RMS departmental templates	Sensitivity labels with matching encryption
Outlook behaviour	Default Outlook label, Do-Not-Forward	Label-policy Outlook settings / advanced settings
Recovery	RMS super-users	Azure RMS super-users (unchanged)

> ***Some AIP add-in features require a newer Office build under built-in labeling, and a few have no exact equivalent.*** *Feature availability and minimum builds are* ***cited or flagged "verify your Office version / licensing"****, never invented. Where a gap has no built-in equivalent, it is recorded as an explicit decision for the label owner — not silently dropped.*

What you can control

- Which users/devices are in the **pilot ring** vs the broad rollout.
- The **Office client policy** (use built-in labeling; disable/allow the AIP add-in) via Group Policy or the Microsoft 365 Apps admin center (Cloud Policy).
- The **label-policy advanced settings** used to reproduce add-in behaviour (default Outlook label, mandatory-labeling posture, custom-permissions availability — verify each key against current docs).
- The **parity mapping** of departmental RMS templates to sensitivity labels.
- The change window and rollback for each cutover ring.

Limitations (be honest with buyers)

- **Licensing- and build-gated.** Sensitivity labeling and its encryption features depend on the tenant's

licensing (Azure Information Protection P1/P2 / Microsoft 365 E3/E5 — **verify your plan**) and on **minimum Office builds** for built-in labeling. Some add-in features (e.g. certain custom-permission or protected-PDF behaviours) need newer builds or have no direct equivalent — this solution surfaces the gap; it cannot license or upgrade for you.

- **The AIP add-in's retirement state and Office ADMX policy names evolve.** Retirement timing, the exact

Group Policy / Cloud Policy setting names, and any registry values are **cited as of authoring and flagged "verify in your tenant / current ADMX"** — never asserted blind.

- **Departmental RMS templates are not auto-converted.** There is no supported cmdlet that turns a

protection template into a sensitivity label; each is reproduced as a label with matching encryption. Whether a legacy template still applies is a records/compliance determination, deferred.

- **This is not a taxonomy or auto-labeling project.** Building new labels, auto-labeling, DLP, or the

Information Protection scanner are separate solutions — this migration preserves the existing estate.

Naming convention (used in every doc and script)

Artifacts this solution creates or captures follow `<Purpose> – Purview – <Stage>` and are reused verbatim across the docs, scripts, and portal.

Artifact	Name
Label / policy / RMS-template inventory export	`AIP/RMS Inventory – Migration – Assess`
AIP-add-in → built-in parity map	`Built-in Labeling Parity – Design`
Label-policy advanced-settings configuration	`Label Policy Config – Configure`
Migration validation / parity report	`Migration Validation – Report`
Migration scope & RMS-health map	`Migration Scope – Map`
Migration pilot ring	`AIP Migration Pilot Ring – Stage1`

The standard deployment model

> *Assess the estate → design the parity map + target client policy → configure label-policy settings and stage the client policy → pilot the cutover on a ring → validate parity against pass/fail gates → expand tenant-wide and decommission the add-in. Pattern:* ***Assess → Design → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — Azure RMS / AIP → Built-in Unified Labeling Migration

Verify **every** requirement before configuring or cutting over. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with

change control) — those are referenced here, not repeated.

2.1 Sensitivity-labeling licensing (labels + encryption)

Description: Licensing that covers **sensitivity labels and label-based encryption** — Azure Information Protection P1/P2 (standalone or via **Microsoft 365 E3/E5** / E5 Compliance). Encryption, custom permissions, and automatic/recommended labeling have specific plan requirements (Global Requirement 1). **Why:** Built-in labeling consumes the same labels the AIP add-in did; the labels and their protection must be licensed for the target users, or the label simply won't apply after cutover. **Verify:** Microsoft 365 admin center → `Billing → Licenses`; run `IPRMSMIG-Assess-Migration.ps1` — it reports the relevant sensitivity-labeling SKUs via Graph. Confirm labels appear in the Purview portal. **Implement:** Assign the required plan to the pilot ring **before** cutover; verify encryption features against your plan — some are E5/E5-Compliance-only.

2.2 Least-privilege information-protection roles

Description: An account in **Information Protection Admin** (or **Compliance Administrator**) to manage sensitivity labels and label policies, plus **Global Administrator** reserved only for one-time actions (activating Azure RMS if it is off, consenting Graph scopes) — per Global Requirement 2. **Why:** Label and policy configuration requires the matching Purview role; the migration itself needs no broad admin beyond the reserved one-time steps. **Verify:** Purview portal → `Settings → Roles & scopes → Role groups` (verify path) — confirm membership in **Information Protection** / **Compliance Administrator**. `Connect-IPPSSession` succeeds. **Implement:** Add the account to the least-privilege role group; reserve a named Global Administrator for the Azure RMS activation and Graph-consent steps only.

2.3 Toolchain: S&C PowerShell + AIPService module + Graph

Description: A workstation with **ExchangeOnlineManagement** (`Connect-IPPSSession` for the label cmdlets), the **AIPService** module (`Connect-AipService` for the Azure Rights Management protection service and its templates), and the **Microsoft Graph** SDK — modern-auth / MFA admin (Global Requirement 3). **Why:** Labels and label policies are managed through S&C PowerShell; RMS activation and protection templates through AIPService; licensing/user data through Graph. Every script uses these. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, AIPService, Microsoft.Graph.Authentication`; `Connect-IPPSSession`, `Connect-AipService`, and `Connect-MgGraph` all succeed. `IPRMSMIG-Assess-Migration.ps1` installs missing modules and reports connectivity. **Implement:** `Install-Module ExchangeOnlineManagement`, `Install-Module AIPService`, and `Install-Module Microsoft.Graph` (`-Scope CurrentUser`); confirm modern-auth sign-in.

2.4 Azure Rights Management (protection) is activated

Description: The **Azure Rights Management** service (the protection behind label encryption and legacy RMS templates) is **activated** for the tenant. **Why:** If the protection service is not activated, label-based encryption and departmental-template protection cannot apply — the migration would drop protection silently. **Verify:** Run `IPRMSMIG-Scope-Health.ps1` — it reports `Get-AipServiceConfiguration FunctionalState` (expect `Enabled`). Purview/portal shows encryption available on labels. **Implement:** If `Disabled`, activate as a one-time Global Admin action (`Enable-AipService` via the AIPService module) under change control; confirm super-users are configured for recovery (2.8).

2.5 Supported Office builds for built-in labeling

Description: The in-scope Office apps are on a **build that supports built-in labeling** and the specific features you use (encryption, co-authoring on encrypted files, custom permissions). Minimum builds are feature-dependent. **Why:** Built-in labeling and several parity features require minimum Office versions; cutting a user over on too old a build breaks parity. **Verify:** Confirm the Office channel/build for the pilot ring against Microsoft's current built-in-labeling capabilities table (**verify — versions evolve**); record it in the scope map. `IPRMSMIG-Test-Parity.ps1` takes the target build as input and flags features that need a newer one. **Implement:** Update the pilot ring to a supported build before cutover; defer users on unsupported builds until updated.

2.6 Current labeling estate inventoried (labels, policies, templates)

Description: A read inventory of the existing **sensitivity labels**, **label policies** (and their advanced settings), and **Azure RMS / protection templates** — the estate that must survive with parity (Global Requirement 4). **Why:** You cannot prove parity against an estate you haven't enumerated; this is the migration's baseline. **Verify:** Run `IPRMSMIG-Assess-Migration.ps1` — it lists labels (`Get-Label`), policies (`Get-LabelPolicy`), and RMS templates (`Get-AipServiceTemplate`), and writes `AIP/RMS Inventory – Migration – Assess`. **Implement:** Produce the inventory before designing the parity map; if labels are missing from the unified store, resolve that (2.7) before cutover.

2.7 Labels present in the unified store (no classic-only labels)

Description: All labels the add-in showed are present and published in the **unified labeling store** (Microsoft Purview), not stranded in a legacy/classic configuration. **Why:** Built-in labeling reads only the unified store; a label that exists only in a legacy location will disappear for users after cutover. **Verify:** `Get-Label` / `Get-LabelPolicy` return every expected label and it is scoped to the pilot users; `IPRMSMIG-Test-Parity.ps1` resolves the effective label set for a pilot user. **Implement:** Publish any missing label in the Purview portal and add it to the pilot users' label policy; run `Execute-AzureADLabelSync` (via `IPRMSMIG-Scope-Health.ps1 -Commit`) so protection picks up label changes.

2.8 Azure RMS super-users configured (recovery)

Description: Super-users are configured in the Azure Rights Management service so protected content remains recoverable during and after the migration. **Why:** If protection or a template is mis-mapped, super-user access is the safety net for recovering encrypted content; it must exist before you change protection behaviour. **Verify:** `IPRMSMIG-Scope-Health.ps1` reports super-user feature state and members (`Get-AipServiceSuperUserFeature` / `Get-AipServiceSuperUser`). **Implement:** Enable the super-user feature and add the named recovery account(s) as a one-time, audited action before cutover — membership determined with security/records stakeholders.

2.9 Office client-policy delivery mechanism

Description: A way to deliver the **Office client policy** that enables built-in labeling and disables the AIP add-in — **Group Policy** (Office ADMX) or the **Microsoft 365 Apps admin center** (Cloud Policy) — scoped so it can target the pilot ring first. **Why:** The cutover is driven by client policy; without a ring-scoped

delivery mechanism you cannot pilot, only flip everyone at once. **Verify:** Confirm you can target a policy to the `AIP Migration Pilot Ring – Stage1` group. The exact setting names / registry values are **flagged "verify against current Office ADMX"** in the Implementation Guide. **Implement:** Prepare a GPO or Cloud Policy configuration scoped to the pilot ring group; do not apply tenant-wide until the ring passes validation.

2.10 Stakeholders, pilot ring, and change control

Description: Engagement with **compliance, records, privacy, security, and information-protection** stakeholders (Global Requirement 5); a defined **pilot ring** (`AIP Migration Pilot Ring – Stage1`) and a **change-control** window for each cutover (Global Requirement 6). **Why:** The cutover changes classification and protection behaviour for real users and touches legacy RMS templates whose applicability is a records/compliance call. **Verify:** Named approvers recorded; the pilot ring group exists and is targetable; a change window and rollback path are agreed. **Implement:** Identify owners; define the ring; agree change control and rollback before the first cutover. Determinations about which legacy templates still apply are the stakeholders' to make.

Checklist

- Sensitivity-labeling licensing (labels + encryption) confirmed for the pilot ring
- Information Protection / Compliance Administrator access; Global Admin reserved for one-time steps
- ExchangeOnlineManagement + AIPService + Microsoft Graph installed; all three connections succeed
- Azure Rights Management activated (`FunctionalState = Enabled`)
- Pilot ring on a supported Office build for the features in use
- Labels, label policies, and RMS templates inventoried (`AIP/RMS Inventory – Migration – Assess`)
- All add-in labels present/published in the unified store and scoped to pilot users
- Azure RMS super-users configured for recovery
- Ring-scoped Office client-policy delivery (GPO or Cloud Policy) prepared
- Stakeholders named; `AIP Migration Pilot Ring – Stage1` defined; change control + rollback agreed

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet*

*detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal** stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Azure RMS / AIP → Built-in Unified Labeling Migration

Follow these steps **in order**. Work is done in the **Microsoft Purview portal** (purview.microsoft.com), **Security & Compliance PowerShell** (`Connect-IPPSSession`, label cmdlets), the **AIPService** module (`Connect-AipService`, the Azure Rights Management protection service), **Microsoft Graph**, and Office **client policy** (Group Policy ADMX or the Microsoft 365 Apps admin center / Cloud Policy). Label-policy changes are previewed with `-WhatIf` before `-Commit`. Artifact names follow the `<Purpose> – Purview – <Stage>` convention from the Overview and are reused verbatim.

> ***Model:*** *Assess the estate → design the parity map + target client policy → configure label-policy settings and stage the client policy →* ***cut the pilot ring over*** *→ validate parity → expand tenant-wide and decommission the add-in. Pattern:* ***Assess → Design → Configure → Pilot → Validate → Expand. The AIP add-in's retirement state, the Office ADMX/Cloud-Policy setting names, and any registry values evolve — verify each against Microsoft's current guidance and your Office channel.*** *Encryption and parity features are licensing-/build-gated — verify against your plan and Office build (Requirements 2.1, 2.5). Cut over the pilot ring only; do not flip the tenant until the ring passes validation.*

Step 1 — Inventory the current estate (Assess)

1. Run `IPRMSMIG-Assess-Migration.ps1 -CsvPath .\AipRmsInventory.csv`. Read-only, it captures: the

sensitivity-labeling SKUs present (Graph), all **sensitivity labels** (`Get-Label` — name, GUID, whether the label applies **encryption**), all **label policies** (`Get-LabelPolicy` — name, scope, and key advanced settings), and the **Azure RMS / protection templates** (`Get-AipServiceTemplate`) with the RMS `FunctionalState`. It writes `AIP/RMS Inventory – Migration – Assess`.

1. In the Purview portal, open **Information Protection** → **Sensitivity labels** and **Label policies**

(verify path) and confirm the on-screen estate matches the inventory.

1. **Pass/fail:** you have a complete, dated inventory of every label, every label policy (with its

advanced settings), and every RMS/protection template — the parity baseline.

Step 2 — Build the parity map (Design)

1. From the inventory, build `Built-in Labeling Parity – Design` — a row per item mapping the **current

experience **to its** built-in-labeling target**:

• **Labels & order** → same labels from the Purview label policy (confirm each is in the unified store,

Requirement 2.7).

• **AIP add-in feature** (default label, mandatory labeling, custom permissions, Do-Not-Forward,

protected-PDF, tracking/revocation) → the **built-in equivalent** (label-policy setting, label encryption setting, or advanced setting) **and the minimum Office build it needs**. Flag any feature with **no direct**

equivalent as an explicit decision for the label owner.

• **Departmental RMS template** → a **sensitivity label with matching encryption** (permissions,

offline access, expiry). There is no auto-convert cmdlet — record the target label and its encryption settings.

1. Record the **target Office client policy** (use built-in labeling; disable the AIP add-in) and the

label-policy advanced settings needed for behavioural parity (e.g. a default Outlook label, mandatory-labeling posture, custom-permissions availability — **verify each key name against current documentation**).

1. **Pass/fail:** every label, feature, and template has a target and a parity decision; gaps are named,

owned, and licensing/build requirements are recorded.

Step 3 — Configure label-policy settings for parity (Configure, preview-first)

1. Dry-run first: `IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<policy>' -SettingsPath .\parity.json -WhatIf`.

With `-WhatIf` (the default) it makes **no changes** — it prints exactly which advanced settings would be applied to which label policy (`Set-LabelPolicy -AdvancedSettings`) and prints the reverse for each.

1. Read every line against `Built-in Labeling Parity – Design`. Confirm the settings reproduce the add-in

behaviour and nothing broadens scope unexpectedly. The `settings` file holds **only** the advanced settings you decided in Step 2 — do not invent keys.

1. Apply to the **pilot policy first**: `IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<pilot policy>'

-SettingsPath .\parity.json -Commit, scoped to the label policy that targets AIP Migration Pilot Ring – Stage1. Writes/updates Label Policy Config – Configure`.

1. **Pass/fail + rollback:** `Get-LabelPolicy -Identity '<policy>'` shows the intended advanced settings.

Rollback re-applies the previous settings (the script prints each prior value) or removes a key.

Step 4 — Confirm labels and protection in the unified store (Configure)

1. Confirm every add-in label is present in the unified store and scoped to the pilot users:

`IPRMSMIG-Test-Parity.ps1 -UserUpn <pilot user> -OfficeBuild '<build>'` resolves the effective label set and flags any label missing for that user.

1. For each **departmental RMS template** in the parity map, confirm the **target sensitivity label**

reproduces its encryption (permissions, offline access, expiry) — recreate the protection on the label in the portal where it is not yet present. Then run `IPRMSMIG-Scope-Health.ps1 -Commit` to `Execute-AzureADLabelSync` so the protection service picks up label changes.

1. **Pass/fail:** every pilot label resolves for a pilot user; each departmental template has a matching,

published protecting label; the label sync has run.

Step 5 — Stage and cut over the pilot ring (Pilot)

1. Stage the **Office client policy** for `AIP Migration Pilot Ring – Stage1` **only** — via Group Policy

(Office ADMX: *use the Sensitivity feature in Office to apply and view sensitivity labels*, and disable the **Azure Information Protection** COM add-in) or the **Microsoft 365 Apps admin center** (Cloud Policy). **Verify the exact setting names / any registry values against the current Office ADMX** — the policy that turns on built-in labeling is commonly delivered as `UseOfficeForLabelling` (verify) plus a managed-add-in setting to disable the add-in.

1. In a **change window**, apply the policy to the ring, have users restart Office, and confirm the add-in is gone and built-in labeling is active (the Sensitivity button appears in the Office ribbon).

1. Keep everyone outside the ring on the current experience. **Pass/fail:** ring devices show built-in labeling, the AIP add-in disabled, and the expected labels in the ribbon.

Step 6 — Validate parity on the ring (Pilot → Validate)

1. Work through the **Testing Guide** on the ring: every label appears and applies; encryption/custom permissions behave as before; the default/mandatory-labeling posture matches; departmental-template protection is reproduced by the mapped label; protected content created on the add-in still opens.

1. Run `IPRMSMIG-Report-Migration.ps1 -CsvPath .\MigrationStatus.csv` to produce `Migration Validation – Report` — the parity matrix with a pass/fail per item.

1. **Pass/fail:** every parity row passes on the ring, or an open gap has a recorded owner decision. Do **not** expand with an unresolved protection/encryption gap.

Step 7 — Expand tenant-wide (Expand)

1. With the ring validated, broaden the Office client policy from the ring to the next rings and then the full population (repeat the change-window + validate pattern per ring). Apply the parity label-policy advanced settings to the broad label policies (`-WhatIf` then `-Commit`).

1. Monitor: re-run `IPRMSMIG-Report-Migration.ps1` after each wave; watch for labels not resolving or

protection failures, and keep RMS super-user recovery available.

1. **Decommission the add-in:** once a population is fully on built-in labeling and validated, retire the AIP add-in for it (managed-add-in policy). Record the completion in `Migration Validation – Report`.

1. **Pass/fail:** the full population is on built-in labeling with parity, the add-in is disabled, and the validation report shows no open protection/encryption gap.

Order of operations

Inventory labels/policies/RMS templates → design parity map + target client policy → dry-run (-WhatIf) label-policy settings → apply to pilot policy → confirm labels + template protection in unified store + sync → stage + cut the pilot ring over → validate parity on the ring → expand ring-by-ring + decommission the add-in

5. Testing Guide — Azure RMS / AIP → Built-in Unified Labeling Migration

Run after building the inventory and after each configuration step, and again after the pilot cutover. Every test is safe (read-only, a `-WhatIf` preview, or a controlled user action on the pilot ring). Validate on `AIP Migration Pilot Ring – Stage1` before expanding tenant-wide.

Test 1 — Inventory is complete and matches the portal

Do: Run `IPRMSMIG-Assess-Migration.ps1`; compare its labels, label policies, and RMS templates against **Information Protection** → **Sensitivity labels / Label policies** in the Purview portal and `Get-AipServiceTemplate`. **Expected:** Every label, policy (with its advanced settings), and RMS/protection template in the portal appears in `AIP/RMS Inventory – Migration – Assess`; nothing is stranded in a classic-only location.

Test 2 — Azure RMS is activated and recoverable

Do: Run `IPRMSMIG-Scope-Health.ps1`. **Expected:** `FunctionalState` reports `Enabled`; the super-user feature state and named recovery account(s) are reported. Protection can apply and encrypted content is recoverable.

Test 3 — Label-policy settings dry-run makes no changes

Do: Run `IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<policy>' -SettingsPath .\parity.json -WhatIf`; afterward run `Get-LabelPolicy -Identity '<policy>'`. **Expected:** The dry-run lists each advanced setting it would apply and prints the prior/reverse value; the policy's advanced settings are unchanged — `-WhatIf` truly changed nothing.

Test 4 — Parity settings resolve after commit

Do: After `-Commit` on the pilot policy, run `Get-LabelPolicy -Identity '<pilot policy>'` (or re-run `IPRMSMIG-Assess-Migration.ps1`). **Expected:** The pilot policy shows exactly the intended advanced settings from `Built-in Labeling Parity – Design` (e.g. default Outlook label, mandatory posture, custom-permissions availability — verified against current key names); no unrelated setting changed.

Test 5 — Every label resolves for a pilot user

Do: Run `IPRMSMIG-Test-Parity.ps1 -UserUpn <pilot user> -OfficeBuild '<build>'`. **Expected:** The pilot user's effective label set matches the add-in label list; no expected label is missing; the script flags any parity feature that needs a newer Office build than the one supplied.

Test 6 — Built-in labeling is active and the add-in is gone (ring)

Do: On a pilot-ring device after the cutover, open Word/Excel/PowerPoint/Outlook and check the ribbon. **Expected:** The **Sensitivity** button (built-in labeling) is present, the expected labels appear in order, and the **Azure Information Protection** add-in is no longer loaded.

Test 7 — Encryption and custom permissions behave as before (ring)

Do: On the ring, apply an encrypting label (and a custom-permissions label if used) to a test document/email; open it as an authorized and an unauthorized user. **Expected:** Protection applies with the same permissions as the add-in did; authorized users open it, unauthorized users are denied; a departmental-template scenario is reproduced by its mapped label.

Test 8 — Content protected by the old add-in / templates still opens

Do: Open documents/emails that were protected **before** the migration (add-in labels and departmental RMS templates) on a migrated ring device. **Expected:** Previously protected content opens with the correct permissions — the migration preserved access; if not, RMS super-user recovery is available and the mapping is corrected before expanding.

Test 9 — Parity report shows no open protection gap

Do: Run `IPRMSMIG-Report-Migration.ps1 -CsvPath .\MigrationStatus.csv`; review `Migration Validation – Report`. **Expected:** Every parity row passes on the ring, or an open gap has a recorded owner decision; **no** encryption/protection row is failing when you decide to expand.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–9 pass on `AIP Migration Pilot Ring – Stage1`: estate inventoried, RMS activated + recoverable, parity settings applied, labels resolve, built-in labeling active with the add-in gone, encryption/templates reproduced, old protected content opens, no open protection gap
Expand pass	Tests 3–9 pass again for each subsequent ring as the Office client policy broadens tenant-wide, with RMS recovery available — until the full population is on built-in labeling and the add-in is decommissioned

6. Troubleshooting Guide — Azure RMS / AIP → Built-in Unified Labeling Migration

Field issues and how to resolve them, in the order you are likely to hit them. Every check here is safe (read-only, a `-WhatIf` preview, or a controlled action on the pilot ring).

After cutover, a label the add-in showed is missing from the Office ribbon

Symptom: A user on built-in labeling no longer sees a label the AIP add-in used to show. **Cause:** The label exists only in a legacy/classic location, or it isn't published to the user in the label policy that applies to them (built-in labeling reads only the **unified store**). **Fix:** Run `IPRMSMIG-Test-Parity.ps1 -UserUpn <user> -ExpectedLabelsPath .\expected-labels.txt` — it flags `MISSING` labels. Publish the label in the Purview portal, add it to the user's label policy, then run `IPRMSMIG-Scope-Health.ps1 -Commit` to sync. Confirm with `Get-LabelPolicy`.

`Connect-AipService` fails or `Get-AipServiceConfiguration` errors

Symptom: The AIPService cmdlets won't connect or return errors. **Cause:** The AIPService module is missing/old, or the account lacks the Azure RMS admin role. **Fix:** `Install-Module AIPService -Scope CurrentUser -Force`; sign in with a modern-auth admin that holds the rights to manage the protection service. Re-run `IPRMSMIG-Scope-Health.ps1`.

Azure RMS `FunctionalState` is `Disabled`

Symptom: `IPRMSMIG-Scope-Health.ps1` reports RMS not `Enabled`; label encryption won't apply. **Cause:** The Azure Rights Management protection service was never activated (or was deactivated). **Fix:** Activate it as a one-time, reserved Global-Admin action (`Enable-AipService`) under change control — this is deliberately **not** done by the scripts. Then confirm super-users are configured (2.8) before changing protection behaviour. Re-run the health check.

Encryption/custom permissions behave differently than under the add-in

Symptom: A label encrypts differently, or a custom-permissions/protected-PDF scenario doesn't work. **Cause:** The parity mapping missed an encryption setting, **or** the user's Office build is too old for that built-in-labeling feature (encryption, co-authoring, custom permissions have minimum builds). **Fix:** Compare the label's encryption settings against the source in `Built-in Labeling Parity – Design`; recreate the missing setting on the label. Run `IPRMSMIG-Test-Parity.ps1 -OfficeBuild '<build>'` — if it flags **NEEDS NEWER BUILD**, update Office. Build thresholds are indicative — **verify against Microsoft's current capabilities table.**

A departmental RMS template's protection didn't carry over

Symptom: Content that used an Azure RMS departmental template is no longer protected as expected. **Cause:** Departmental/protection templates are **not auto-converted** to labels — the mapped label wasn't created with matching encryption, or wasn't published. **Fix:** For each template in the inventory (`Get-AipServiceTemplate`), create/confirm a sensitivity label reproducing its permissions, offline access, and expiry; publish it to the affected users; run `Execute-AzureADLabelSync` (`IPRMSMIG-Scope-Health.ps1 -Commit`). Whether the template still applies is a records/compliance determination — confirm with the owner.

The AIP add-in is still loading after you applied the client policy

Symptom: Users still see the **Azure Information Protection** add-in in the ribbon post-cutover. **Cause:** The client policy (disable the add-in / enable built-in labeling) didn't reach the device, or the wrong setting/registry value was used — the Office ADMX names and any registry values evolve. **Fix:** Confirm the GPO / Cloud Policy targets the user's device and Office restarted. **Verify the exact setting names and any registry value against the current Office ADMX** (the built-in-labeling switch is commonly `UseOfficeForLabelling`, plus a managed-add-in setting to disable the add-in) — do not assume a value from an old screenshot.

`Set-LabelPolicy -AdvancedSettings` rejects a key or has no effect

Symptom: `IPRMSMIG-Set-LabelPolicy.ps1 -Commit` errors on a key, or the behaviour doesn't change. **Cause:** The advanced-setting **key name** is wrong or deprecated — these vary and evolve; or the setting applies to a different scope (Outlook vs client) than expected. **Fix:** Verify the key name against Microsoft's **current** label-policy advanced-settings documentation and correct the `parity.json`. Re-run with `-WhatIf` first. The script never invents keys — it applies only what you supply.

Old protected content won't open on a migrated device

Symptom: Documents/emails protected before the migration are denied on a cutover device. **Cause:** A protection mapping is wrong, or the user genuinely lacks rights. **Fix:** Use **RMS super-user** recovery (confirmed in 2.8) to open and re-protect the content while you correct the label/template mapping; then re-validate with the Testing Guide before expanding. This is exactly why super-users must exist before the first cutover.

`Execute-AzureADLabelSync` runs but protection still lags

Symptom: You synced but the user still sees the pre-change behaviour. **Cause:** Label/protection changes take time to propagate after a sync, and Office caches label policy. **Fix:** Allow propagation time, have the user restart Office (which refreshes the label policy), and re-test. If it persists, re-check the label is published to that user and RMS is `Enabled`.

When to stop and escalate

- **Azure RMS won't activate** or a plan doesn't include the required encryption feature → this is a licensing/tenant action; record it and do not force protection changes until it's resolved.
- A **records/retention or legal determination** about whether a legacy departmental template still applies is being asked of you → escalate to compliance, records, privacy, or legal. This solution provides the technical mapping and evidence, never the determination.
- A **client-policy setting name or registry value in this guide doesn't match your environment** → the Office ADMX and the AIP add-in retirement state are evolving; verify the current value against Microsoft's guidance and your Office channel rather than assuming the step is wrong.

7. Best Practices — Azure RMS / AIP → Built-in Unified Labeling Migration

- **Migrate for parity, not just for cutover.** The deliverable is that users see the same labels,

encryption, and behaviour after the move. Build the parity map first and make every label, add-in feature, and RMS template map to a target — a silently dropped feature is unprotected data, not a cosmetic change.

- **Cut over a pilot ring in a change window before anyone else.** Flipping the labeling client changes

how every document and email is classified and protected. Prove parity on `AIP Migration Pilot Ring – Stage1` first; it is cheap to fix on a ring and expensive to fix tenant-wide.

- **Confirm Azure RMS is activated and super-users exist before you touch protection.** Activation is the

prerequisite for label encryption and departmental-template protection; super-users are the recovery net if a mapping is wrong. Both must be in place before the first cutover.

- **Always dry-run label-policy changes.** Run `IPRMSMIG-Set-LabelPolicy.ps1 -WhatIf` and read every line

before `-Commit`; the script prints the prior value for each advanced setting so rollback is trivial. Never guess an advanced-setting key name — verify it against current documentation.

- **Treat departmental RMS templates as manual conversions.** There is no supported cmdlet that turns a

protection template into a sensitivity label. Recreate each as a label with matching encryption and validate it opens old content — don't assume the template "just carries over."

- **Verify Office builds before cutting a ring over.** Built-in labeling and several parity features need

minimum Office versions; a user on too old a build loses a feature at cutover. Update the ring first and let `IPRMSMIG-Test-Parity.ps1` flag features that need a newer build.

- **Verify the AIP add-in's retirement state and the client-policy setting names against current docs.**

Retirement timing, the Office ADMX / Cloud Policy setting names, and any registry values evolve — cite as of authoring and confirm in your tenant rather than trusting a value from last quarter.

- **Run** `Execute-AzureADLabelSync` **after label/protection changes.** Protection picks up label changes on

a sync; running it removes "the label works in the portal but not for the user" surprises during validation.

- **Keep the AIP add-in until parity is proven, then decommission deliberately.** Disable the add-in for a

population only after that population is validated on built-in labeling — and record the decommission in the validation report so the estate's state is auditable.

- **This is a migration, not a taxonomy build.** Do not redesign labels, add auto-labeling, or stand up

the Information Protection scanner here — hand off to those dedicated Purview solutions once the estate is safely on built-in labeling. Which legacy templates still apply is a records/compliance determination, not a technical default.

8. Deliverables Summary — Azure RMS / AIP → Built-in Unified Labeling Migration

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The AIP add-in retirement, the migration to built-in unified labeling with policy parity, legacy Azure RMS / departmental-template mapping, the parity dimensions, limitations, and the naming convention
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered portal + S&C PowerShell + AIPService + Graph + client-policy steps (inventory, parity map, apply label-policy settings pilot-first, confirm labels/protection, cut the ring over, validate, expand + decommission)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`IPRMSMIG-Assess-Migration.ps1`	Read-only estate inventory: sensitivity-labeling SKUs (Graph), all sensitivity labels + whether each encrypts (`Get-Label`), all label policies + key advanced settings (`Get-LabelPolicy`), Azure RMS templates and `FunctionalState` (`Get-AipServiceTemplate`). Writes `AIP/RMS Inventory – Migration – Assess`
`IPRMSMIG-Set-LabelPolicy.ps1`	Label-policy advanced-settings helper for behavioural parity: applies the settings from a JSON file to a named label policy via `Set-LabelPolicy -AdvancedSettings`; `-WhatIf` **by default**, `-Commit` to apply; prints the prior value for each key. Writes `Label Policy Config – Configure`

Script	What it does
`IPRMSMIG-Test-Parity.ps1`	Read-only parity check: resolves the effective label set for a pilot user, flags labels missing from the unified store, and flags parity features that need a newer Office build than the one supplied
`IPRMSMIG-Report-Migration.ps1`	Read-only migration/parity report: builds the parity matrix (labels, add-in features, RMS templates → built-in target, with a pass/fail per row) and exports it. Writes `Migration Validation – Report`
`IPRMSMIG-Scope-Health.ps1`	Read-only scope/health helper: reports Azure RMS activation (`Get-AipServiceConfiguration`), super-user recovery posture, pilot-scope readiness, and toolchain connectivity; with `-Commit` triggers `Execute-AzureADLabelSync`. Writes `Migration Scope – Map`

All scripts are **safe by default** — read-only, or a `-WhatIf` preview requiring an explicit `-Commit` to change anything. No credentials or secrets are stored in code; every script uses interactive modern-auth (`Connect-IPPSSession` / `Connect-AipService` / `Connect-MgGraph`).

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Your labeling estate moved off the retiring AIP add-in and off legacy Azure RMS departmental templates onto **built-in unified labeling** in Microsoft Purview — **with parity**: the same labels, the same encryption and custom permissions, the same Outlook and mandatory-labeling behaviour, and previously protected content still opening — piloted on a ring, validated against pass/fail gates, then expanded tenant-wide with the add-in decommissioned. The capability is in place, defensible, and evidenced.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPRMSMIG-Assess-Migration.ps1

Purpose: Builds the migration baseline: which sensitivity-labeling SKUs the tenant has (via Graph), all sensitivity labels and whether each applies encryption (Get-Label), all label policies and their key advanced settings (Get-LabelPolicy), and the Azure Rights Management templates plus the protection service FunctionalState (AIPService). Read-only — makes no changes. Writes the export "AIP/RMS Inventory - Migration - Assess". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Assess-Migration.ps1 -CsvPath .\AipRmsInventory.csv PowerShell 5.1+ or 7+. Requires ExchangeOnlineManagement (Connect-IPPSSession), AIPService (Connect-AipService), and the Microsoft Graph SDK; Graph read scope: Organization.Read.All. EXPECTED RESULTS A console summary of labeling SKUs, labels (name/GUID/encrypts), label policies (+ advanced settings), and RMS templates with RMS FunctionalState, plus an optional CSV. PASS = a complete, dated inventory of every label, policy, and template to design parity against. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only. No secrets in code. Sensitivity labeling is licensing-dependent and the portal/AIP retirement state evolves — verify capability availability and the AIP add-in status against your tenant and Microsoft's current docs. > [CmdletBinding()] param([string]$CsvPath, [string]$ReportName = 'AIP/RMS Inventory - Migration - Assess')

How to run: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Assess-Migration.ps1 -CsvPath .\AipRmsInventory.csv PowerShell 5.1+ or 7+. Requires ExchangeOnlineManagement (Connect-IPPSSession), AIPService (Connect-AipService), and the Microsoft Graph SDK; Graph read scope: Organization.Read.All. EXPECTED RESULTS A console summary of labeling SKUs, labels (name/GUID/encrypts), label policies (+ advanced settings), and RMS templates with RMS FunctionalState, plus an optional CSV. PASS = a complete, dated inventory of every label, policy, and template to design parity against. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only. No secrets in code. Sensitivity labeling is licensing-dependent and the portal/AIP retirement state evolves — verify capability availability and the AIP add-in status against your tenant and Microsoft's current docs. > [CmdletBinding()] param([string]$CsvPath, [string]$ReportName = 'AIP/RMS Inventory - Migration - Assess')

IPRMSMIG-Report-Migration.ps1

Purpose: Produces the migration status report: joins the current unified-store estate (labels, label policies, RMS templates) to your parity map and emits a per-row PASS/FAIL matrix — label present, encryption reproduced, template mapped, policy setting applied. Read-only — makes no changes. Writes "Migration Validation - Report". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Report-Migration.ps1 -ParityMapPath .\parity-map.csv -CsvPath .\MigrationStatus.csv -ParityMapPath = the parity map you built (CSV columns: Kind, SourceItem, TargetLabel, ParityDecision). PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession); optional AIPService for template status. If -ParityMapPath is omitted, the report lists the live estate for review. EXPECTED RESULTS A parity matrix (console + optional CSV) with a Result per row. PASS = every parity row resolves against the live estate; no encryption/protection row is failing. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only. No secrets in code. The report evidences parity; whether a legacy template still applies is a records/compliance determination, deferred to the relevant

governance function. > [CmdletBinding()] param([string]$ParityMapPath, [string]$CsvPath, [string]$ReportName = 'Migration Validation - Report')

How to run: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Report-Migration.ps1 -ParityMapPath .\parity-map.csv -CsvPath .\MigrationStatus.csv -ParityMapPath = the parity map you built (CSV columns: Kind, SourceItem, TargetLabel, ParityDecision). PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession); optional AIPService for template status. If -ParityMapPath is omitted, the report lists the live estate for review. EXPECTED RESULTS A parity matrix (console + optional CSV) with a Result per row. PASS = every parity row resolves against the live estate; no encryption/protection row is failing. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only. No secrets in code. The report evidences parity; whether a legacy template still applies is a records/compliance determination, deferred to the relevant governance function. > [CmdletBinding()] param([string]$ParityMapPath, [string]$CsvPath, [string]$ReportName = 'Migration Validation - Report')

IPRMSMIG-Scope-Health.ps1

Purpose: Confirms the protection foundation before/after cutover: Azure Rights Management activation (Get-AipServiceConfiguration FunctionalState), super-user recovery posture (Get-AipServiceSuperUserFeature / Get-AipServiceSuperUser), toolchain connectivity, and the pilot scope. Read-only by default; with -Commit it runs Execute-AzureADLabelSync so the protection service picks up label/protection changes. Writes "Migration Scope - Map". HOW TO RUN Read-only health check: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Scope-Health.ps1 -CsvPath .\MigrationScope.csv Also trigger a label sync (benign, one-way sync of label changes to protection): powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Scope-Health.ps1 -Commit PowerShell 5.1+/7+. Requires AIPService (Connect-AipService) and ExchangeOnlineManagement (Connect-IPPSSession, for Execute-AzureADLabelSync). EXPECTED RESULTS RMS FunctionalState, super-user feature state + members, and connectivity reported. PASS = RMS Enabled and recoverable (super-users configured) before you change protection behaviour. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only unless -Commit is passed (which only triggers a label sync). No secrets in code. Activating Azure RMS itself is a separate, reserved Global-Admin action (Enable-AipService) and is intentionally NOT performed here. > [CmdletBinding()] param([switch]$Commit, [string]$CsvPath, [string]$ReportName = 'Migration Scope - Map')

How to run: Read-only health check: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Scope-Health.ps1 -CsvPath .\MigrationScope.csv Also trigger a label sync (benign, one-way sync of label changes to protection): powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Scope-Health.ps1 -Commit PowerShell 5.1+/7+. Requires AIPService (Connect-AipService) and ExchangeOnlineManagement (Connect-IPPSSession, for Execute-AzureADLabelSync). EXPECTED RESULTS RMS FunctionalState, super-user feature state + members, and connectivity reported. PASS = RMS Enabled and recoverable (super-users configured) before you change protection behaviour. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only unless -Commit is passed (which only triggers a label sync). No secrets in code. Activating Azure RMS itself is a separate, reserved Global-Admin action (Enable-AipService) and is intentionally NOT performed here. > [CmdletBinding()] param([switch]$Commit, [string]$CsvPath, [string]$ReportName = 'Migration Scope - Map')

IPRMSMIG-Set-LabelPolicy.ps1

Purpose: Reproduces the AIP add-in's behaviour on built-in labeling by applying the advanced settings you decided in the parity map (default Outlook label, mandatory-labeling posture, custom-permissions availability, etc.) to a named label policy via Set-LabelPolicy -AdvancedSettings. SAFE BY DEFAULT — previews with -WhatIf and changes nothing until you pass -Commit. Prints the current value of each key first so rollback is trivial. Writes "Label Policy Config - Configure". HOW TO RUN Dry-run (no changes): powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<policy>' -SettingsPath .\parity.json -WhatIf Apply: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<policy>' -SettingsPath .\parity.json -Commit SettingsPath = a JSON file of { "Key": "Value", ... } advanced settings you verified against current documentation. PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS Dry-run lists each advanced setting it would apply and its current value; nothing changes. With -Commit, Get-LabelPolicy shows the intended advanced settings. PASS = the policy's behaviour matches the approved parity map; no unrelated setting changed. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Advanced-setting KEY NAMES vary and evolve — supply only keys you verified against Microsoft's current docs; the script does not invent them. Apply to the pilot policy first. No secrets in code. > [CmdletBinding(DefaultParameterSetName='WhatIf')] param([Parameter(Mandatory)][string]$PolicyName, [Parameter(Mandatory)][string]$SettingsPath, [switch]$Commit, [switch]$WhatIf = $true, [string]$ReportName = 'Label Policy Config - Configure', [string]$CsvPath)

How to run: Dry-run (no changes): powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<policy>' -SettingsPath .\parity.json -WhatIf Apply: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Set-LabelPolicy.ps1 -PolicyName '<policy>' -SettingsPath .\parity.json -Commit SettingsPath = a JSON file of { "Key": "Value", ... } advanced settings you verified against current documentation. PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession). EXPECTED RESULTS Dry-run lists each advanced setting it would apply and its current value; nothing changes. With -Commit, Get-LabelPolicy shows the intended advanced settings. PASS = the policy's behaviour matches the approved parity map; no unrelated setting changed. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Advanced-setting KEY NAMES vary and evolve — supply only keys you verified against Microsoft's current docs; the script does not invent them. Apply to the pilot policy first. No secrets in code. > [CmdletBinding(DefaultParameterSetName='WhatIf')] param([Parameter(Mandatory)][string]$PolicyName, [Parameter(Mandatory)][string]$SettingsPath, [switch]$Commit, [switch]$WhatIf = $true, [string]$ReportName = 'Label Policy Config - Configure', [string]$CsvPath)

IPRMSMIG-Test-Parity.ps1

Purpose: Validates built-in-labeling parity for a pilot user: resolves the label policies that apply to the user and the effective set of labels, flags any expected label missing from the unified store, and flags parity features that need a newer Office build than the one supplied. Read-only — makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Test-Parity.ps1 -UserUpn user@contoso.com -OfficeBuild '16.0.17000' -ExpectedLabelsPath .\expected-labels.txt -ExpectedLabelsPath (optional) = a text file of the label display names the AIP add-in showed this user, one per line, to check nothing dropped. PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession); optional Microsoft Graph SDK for group-membership resolution. EXPECTED RESULTS A per-label PASS/MISSING list for the user and a feature-vs-build advisory. PASS = the user's effective labels match the expected set and no in-use feature needs a build newer than -OfficeBuild. .NOTES Purview — Azure RMS / AIP to Built-in

Unified Labeling Migration. Read-only. Minimum Office builds for built-in-labeling features EVOLVE — the build thresholds below are indicative and must be verified against Microsoft's current capabilities table. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$UserUpn, [string]$OfficeBuild, [string]$ExpectedLabelsPath, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\IPRMSMIG-Test-Parity.ps1 -UserUpn user@contoso.com -OfficeBuild '16.0.17000' -ExpectedLabelsPath .\expected-labels.txt -ExpectedLabelsPath (optional) = a text file of the label display names the AIP add-in showed this user, one per line, to check nothing dropped. PowerShell 5.1+/7+. Requires ExchangeOnlineManagement (Connect-IPPSSession); optional Microsoft Graph SDK for group-membership resolution. EXPECTED RESULTS A per-label PASS/MISSING list for the user and a feature-vs-build advisory. PASS = the user's effective labels match the expected set and no in-use feature needs a build newer than -OfficeBuild. .NOTES Purview — Azure RMS / AIP to Built-in Unified Labeling Migration. Read-only. Minimum Office builds for built-in-labeling features EVOLVE — the build thresholds below are indicative and must be verified against Microsoft's current capabilities table. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$UserUpn, [string]$OfficeBuild, [string]$ExpectedLabelsPath, [string]$CsvPath)